

Trustpilot Agentic GraphRAG Design Document

DOC TYPE: DESIGN DOCUMENT & POC · DATE: OCTOBER 4, 2026

1. Executive Summary

This document outlines a Proof of Concept (POC) for extracting actionable user interests from unstructured Trustpilot reviews. By moving beyond rigid keyword extraction and adopting an Agentic GraphRAG architecture, the system will autonomously extract relational data, group it into thematic communities, and allow AI agents to navigate the resulting knowledge graph to answer complex business queries.

2. Data Schema & Ingestion

The system will ingest the provided 2021 review dataset utilizing the following schema structure to establish the foundational nodes and edges of the graph:

- **Review_id:** Unique identifier (serves as the Primary Key for source nodes).
- **Created_date:** Used for temporal edge weighting and time-series filtering.
- **Star rating:** Provides sentiment validation across defined communities.
- **Review title & Review text:** The primary unstructured text used for entity extraction.
- **Review source:** Metadata for segmenting organic vs. invited reviews.
- **Language:** Enables multilingual routing and localization clustering.

3. Pipeline Architecture

Phase 1: Entity & Triplet Extraction (Updated Layer 1)

Instead of matching flat keywords, a Large Language Model (LLM) will process the review text to extract Aspect-Opinion-Sentiment triplets. This allows the system to capture the precise context of a user's statement. To optimize for short-form consumer reviews, this layer now utilizes:

- **Contextual String Assembly:** Metadata is prepended to the review body to anchor the LLM's understanding without requiring arbitrary text splitting.
- **Single-Pass Joint Extraction:** The LLM extracts both the macro-topics (Aspects) and the micro-relationships (Triplets) simultaneously in one forward pass.
- **Runtime Schema Enforcement:** The LLM's output is strictly constrained using a Pydantic schema, locking predicates to predefined Enums to prevent graph fracturing.
- **Post-Extraction Canonicalization:** Extracted subjects and objects are normalized and fuzzy-matched to merge spelling variants into unified canonical nodes.

Phase 2: Community Detection & Interest Definition

The extracted triplets form a massive Knowledge Graph. We define macro and micro "Interests" not by single keywords, but by semantic clusters of related concepts.

- **Clustering Algorithm:** Leiden or Louvain community detection groups highly interconnected nodes into distinct clusters (e.g., grouping "Cart," "Credit Card," and "Apple Pay").

- **Interest Summarization:** An LLM reads the entities within a cluster and generates a human-readable summary.

Phase 3: Downstream Applications (Decomposed Layer 3)

Because the GraphRAG architecture fundamentally decouples the semantic data layer from the application layer, the enriched Knowledge Base serves as a centralized hub for three parallel downstream operational tracks. This transforms the system from a standalone conversational interface into a comprehensive semantic data warehouse.

Track 1: Agentic Flows (Conversational AI)

Users query the system in natural language, and a routing LLM acts as the orchestrator to determine the optimal graph execution path based on the user's intent:

- **Global Search (Map-Reduce):** Deployed for thematic, open-ended questions (e.g., *"What are the top 3 complaints from 1-star reviews?"*). The system aggregates and synthesizes the pre-computed Community Summaries.
- **Local Search (Parameterized Cypher):** Deployed for specific root-cause drill-downs (e.g., *"Why is Apple Pay failing?"*). To prevent Text-to-Cypher hallucinations, the agent performs Named Entity Recognition (NER) and injects the extracted parameters into hardcoded, pre-validated Cypher templates rather than writing queries from scratch.

Track 2: BI & Analytics (Visual Graph Exploration)

Human analysts and product managers can bypass the LLM reasoning loop entirely to explore the data structurally.

- **Direct Visualization:** Using enterprise tools like Neo4j Bloom or Linkurious, teams can visually query the graph database, filter nodes by metadata (e.g., 1-star ratings, specific temporal bounds), and manually expand relationships to uncover how a specific software bug connects to negative sentiment across different user demographics.

Track 3: ML & Graph Neural Networks (Predictive Intelligence)

Instead of solely analyzing historical data, this track leverages the graph's mathematical topology to forecast future user behavior and systemic risks.

- **Node Classification:** Graph Neural Networks (e.g., GraphSAGE) assess node degree, connectivity, and structural proximity to known "complaint" clusters. This allows the system to flag a seemingly neutral review as a high churn risk based purely on its structural placement within the network.
- **Link Prediction:** The network forecasts emerging issues by predicting edge formations between currently unconnected nodes (e.g., predicting that a recent UI update node will shortly connect to a surge in customer support ticket nodes before the correlation is explicitly stated in reviews).

4. Example Agentic Workflow

An autonomous agent, utilizing a ReAct (Reason + Act) framework, will handle complex investigative tasks without human intervention. The table below illustrates a standard trace execution:

Phase	Action / Input	System Process & Observation
1. Prompt	User requests: "Identify the most damaging emerging interest from 2021."	The Agent initializes the reasoning loop and analyzes available graph tools.
2. Global Query	Agent calls Query_Global_Communities filtered by 1 and 2-star ratings.	The Graph returns Cluster 12. The summarized interest is identified as: "Unexpected auto-renewal charges."
3. Drill-down	Agent calls Query_Local_Entity on the "auto-renewal" node to find root causes.	The Graph retrieves connected edges: (auto-renewal) <- [CAUSED_BY] (hidden toggle switch).
4. Synthesis	Agent drafts the final explanatory response.	Combines the macro-trend (auto-renewal complaints) with the specific UI root cause (hidden toggles) and delivers the final text to the user.

Next Steps for Implementation: Configure the LLM prompts for reliable triplet extraction, establish the vector database/graph store (e.g., Neo4j), and define the LangChain toolsets for the routing agent.

5. POC to Production Steps

The migration from an experimental local POC to an enterprise production deployment follows these structured steps:

- **Step 1: Schema Bootstrapping:** Run unconstrained extraction across a small review sample to observe organic terminology, then freeze the Pydantic Enum schema with strict validation rules.
- **Step 2: Component Alignment:** Migrate from local embedded tools (Kùzu, ChromaDB, cdlib) to horizontally scalable cloud microservices (Neo4j Enterprise, Pinecone, Neo4j Graph Data Science).
- **Step 3: Constraint Hardening:** Prevent Text-to-Cypher hallucinations by restricting the agent to Named Entity Recognition and injecting parameters into pre-validated, hardcoded query templates. Implement asynchronous message queues for batch ingestion.
- **Step 4: Continuous Evaluation:** Schedule periodic offline Leiden re-clustering runs to detect emerging shifts in user interests over time.